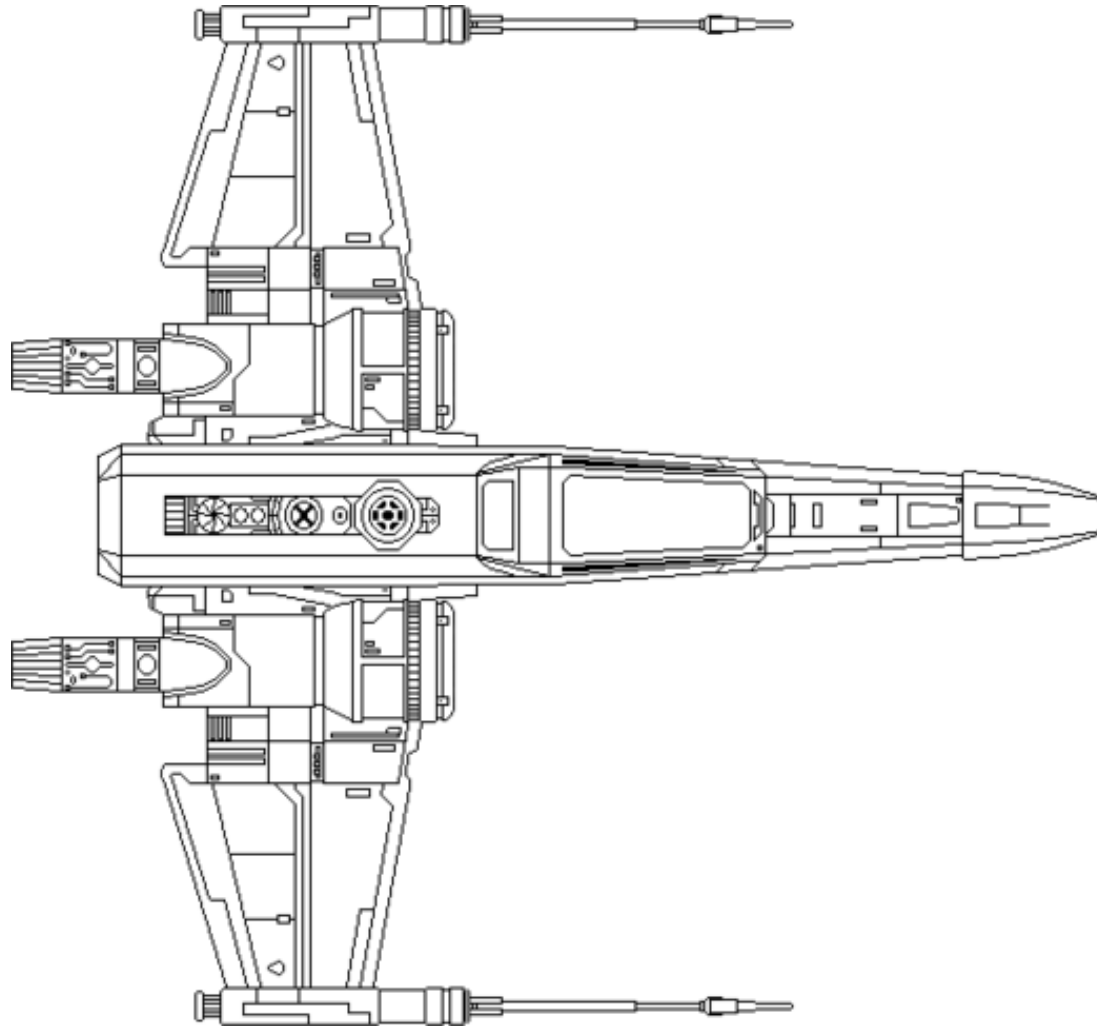


beginning_of_file

X-Wing Advisor (XWA)



XWA: The Requirements

- Demonstrate responsive UI design
- Demonstrate the value of data visualizations
- Demonstrate best-practice information assurance and security
- Demonstrate Minimal API architecture
- Demonstrate test harness in the orchestration layer
- Architect the solution with the lightest deployment footprint

XWA: Architecture

- .NET 8
- Modular Monolith
 - XWA.UI
 - XWA.WebAPI
 - XWA.Core
- HTML 5
- Minimal API
- AngularJS responsive user experience
- D3 (“Data-Driven Documents”)
 - Open-source JavaScript data visualization library
 - Written by Mike Bostock ([u/mbostock](https://github.com/mbostock))
 - <https://d3js.org/what-is-d3>

XWA.UI: The Presentation Layer Project

- C#
 - **Program.cs**: The application launch mechanism
 - **UserRequest.cs**: The model class for user for authentication
 - **SessionFactory.cs**: The model class for persisting the user, the JWT Bearer token, and application version information
- HTML5
 - Files are located under **wwwroot/app**
 - Launched via **Program.cs**
 - **WebApplication.UseDefaultFiles()**
 - **WebApplication.UseStaticFiles()**

XWA.WebAPI: The Orchestration Layer Project

- Classes are organized under the **Features** folder, according to their use-case
- Controller classes are replaced with endpoint classes
 - **IEndpointRouteBuilder.MapGet("feature/code")**
- Endpoints get wired up using dependency injection
- Dependency injection gets wired up via the **ServiceCollection**
 - **IHostApplicationBuilder.Services.AddScoped<IGumby, Gumby>()**
- DTO model classes are replaced with **Request** and **Response** model classes
- Classes do not adhere to a strict CQRS (Command Query Responsibility Segregation) pattern

XWA.WebAPI: The Orchestration Layer Project

- **ServiceExtension.cs** wires up most critical functionality
 - CORS (Cross-Origin Resource Sharing) gate-keeping
 - **DbContext**, where applicable
 - JWT Bearer **TokenProvider**
 - Options pattern functionality for **appsettings** configuration items
 - Validators
 - Exception handlers
 - Swashbuckle **Swagger** test harness and documentation
 - Authorization hook
 - Authentication hook

XWA.WebAPI: The Orchestration Layer Project

- **Program.cs** adds the critical functionality wired up from **ServiceExtension.cs** to the application
 - CORS
 - Routing
 - Authorization
 - Authentication
 - HTTP redirection
 - **Swagger** availability
 - Exception handling
 - Endpoint mapping

XWA.Core: The Shared Library Project

- Contains app-specific helper classes, constants classes, and utility classes
- Cuts across all domain boundaries
- Very lightweight

XWA: The (Missing) Database Project

- Data is read from CSV files
- CSV files use **FileSystemObject** (FSO) .NET routines in the **CsvHelper** NuGet package
- The path location to the CSV files is stored in the **appsettings** JSON file
- Since reading **appsetting** JSON files is non-performant, the Options pattern is used when the application launches, making configuration settings available through dependency injection throughout the life of the application
- Since reading CSV files is non-performant, data is read in the optimal order to minimize negative performance impacts

XWA: JWT Bearer Token Functionality

demo

XWA: Wrap-Up

questions

end_of_file